

GreenRAN - Documentação Consolidada

Gerado em 2026-05-08 12:00:29

Este PDF reúne a visão consolidada do projeto, a trilha article00, os aplicativos App1/App2/App3, a arquitetura veicular em CARLA 2D, a hierarquia do rApp e os caminhos futuros de adaptação para ML e DRL.

GreenRAN O-RAN - Coordenação rApp-xApps

Leitura Recomendada Atual

Para o estado atual do projeto, estes documentos devem ser lidos primeiro:

- `docs/NS3\_ARTICLE00\_TEMPORAL\_PIPELINE.md`
- fluxo oficial para rodar a metodologia temporal do `article00` sobre dados coletados do `ns-3`;
- wrapper unico de simulacao + coleta + conversao + treino temporal;
- artefatos finais em `runs/ns3\_article00/`.
- `docs/APP1\_APP2\_ANALISE.md`
- comparacao objetiva entre `App1-Vigilancia` e `App2-Monitoramento`;
- grau de maturidade de cada app;
- integracao com GreenRAN e papel atual no `rApp`.
- `docs/CARLA\_NS3\_INTEGRACAO.md`
- plano tecnico para integrar `CARLA + ns-3 + GreenRAN`;
- separacao entre simulacao fisica, simulacao de rede e politica de decisao;
- primeira iteracao recomendada para veiculos conectados.
- `docs/CARLA\_2D\_MODELO\_VEICULAR.md`
- modelo 2D exato para a trilha veicular;
- estado minimo por veiculo;
- uso por `App3`, `xApp-VehicleSafety` e `rApp`.
- `docs/APP3\_VEICULAR\_ARQUITETURA.md`
- arquitetura proposta para um novo `App3-Veicular`;
- separacao entre app, xApp e `rApp`;
- backend, snapshot e dados minimos por veiculo.
- `docs/CONFLICT\_DATASET\_PIPELINE.md`
- exportacao do dataset de conflitos;
- aprendizado da matriz operacional;
- treino GraphSAGE por epocas;
- estrutura da coleta por cenarios.
- `docs/ANALISE\_CONFLITOS\_GNN.md`
- resumo do treino multiseed;
- splits usados em `conflito\_implicito` e `recuperacao`;
- os 6 graficos finais e o que eles significam para o cenario.
- `docs/DOCUMENTACAO\_GREENRAN.md`
- documentacao ampla do sistema.
- `docs/DRL\_RUNTIME\_MELHORIAS.md`
- melhorias no predictor DRL em runtime;
- trace, warmup, estabilidade e leitura correta do dashboard;

- ajuste de oscilacao do simulador do App2 sem mexer na politica do `rApp`.
- `docs/ML\_RUNTIME\_MELHORIAS.md`
- correcoes causais no predictor de ML;
- retreino com split temporal;
- poda de features enviesadas;
- validacao em runtime saudavel sem bloqueio espurio.

Visão Geral

O projeto GreenRAN implementa um sistema de coordenação entre rApps (Non-RT RIC) e xApps (Near-RT RIC) para gerenciamento de energia em redes O-RAN, garantindo que câmeras de vigilância 4K nunca sofram latência alta.

Arquitetura de Coordenação

Hierarquia de Controle

...

[illegible]

■ AROUITETURA GREENRAN ■

[illegible]

■ ■

■ rApp Orchestrator (Juiz) ■

■ ■ ■ ■ Visibilidade GLOBAL (Data Lake) ■

■ ■■■■ Calcula: CVaR/UE, Slope, Variância ■

■ ■ ■ ■ Decide: ALLOWED/BLOCKED/CONDITIONAL ■

■ ■■■■ Envia: Comandos para xApps ■

□ □

■ xApp Slicer (Advogado 1) ■

■ ■■■■ Monitora P95 latency por UE ■

■ ■ ■ ■ Se câmeras > 80ms: CRITICAL → aloca PRBs ■

■ ■■■■ rApp decide sobre energia ■

■ ■

■ xApp Energy Saver (Advogado 2) ■

■ ■ ■ ■ Lê comandos do rApp ■

■■■■ Executa: FULL POWER/REDUCE/POWER DOWN ■

■ ■ ■ ■ Watchdog: 5s → FULL POWER ■

■ ■

...

Regras de Coordenação

REGRA 1: Prioridade Máxima - Câmeras

...

Se Slicer CRITICAL:

■■■■ rApp: BLOCKED

■■■■ Energy Saver: FULL_POWER

■■■■ Motivo: Proteger câmeras (prioridade absoluta)

...

REGRA 2: Proteção de SLA

...

Se CVaR $\geq$ 80ms:

■■■■ rApp: BLOCKED

■■■■ Energy Saver: FULL_POWER

■■■■ Motivo: SLA em risco

...

REGRA 3: Prevenção de Problemas

...

Se Slope > 2ms/s:

■■■■ rApp: BLOCKED (preventivo)

■■■■ Energy Saver: FULL_POWER

■■■■ Motivo: Latência vai subir

...

REGRA 4: Economia Segura

...

Se CVaR < 60ms E Slope < 0:

■■■■ rApp: ALLOWED

■■■■ Energy Saver: POWER_DOWN

■■■■ Motivo: Rede estável e melhorando

...

Zonas de Operação

Zona Verde (Normal)

...

Condição: CVaR < 60ms

Decisão: ALLOWED

Ação: Energy Saver pode economizar

Comportamento: Sistema funcionando normalmente

...

Zona Amarela (Prevenção)

...

Condição: $60\text{ms} < \text{CVaR} < 80\text{ms}$

Decisão: CONDITIONAL

Ação: Monitorar, não reduzir

Comportamento: rApp monitora tendência

...

Zona Vermelha (Crítico)

...

Condição: $\text{CVaR} > 80\text{ms}$

Decisão: BLOCKED

Ação: FULL_POWER, não economizar

Comportamento: SLA em risco, proteger câmeras

...

Dashboard de Coordenação

Painéis Disponíveis

...

CVaR/UE: Mostra CVaR atual e decisão

Slope: Mostra tendência (subindo/descendo)

Decisão: Mostra ALLOWED/BLOCKED/CONDITIONAL

Regras: Tabela com status de cada regra

...

Como Interpretar

...

$\text{CVaR} < 60\text{ms} + \text{Slope} < 0$: REGRA 4 ATIVA → Energia pode economizar

$\text{CVaR} > 80\text{ms}$: REGRA 2 ATIVA → SLA em risco

Slicer CRITICAL: REGRA 1 ATIVA → Câmeras prioridade

...

Implementação Técnica

rApp Orchestrator

```
```python
```

```
def make_decision(self, slicer_intent, energy_intent):
```

```
 """
```

Regras de coordenação rApp-xApps:

1. Câmeras são prioridade MÁXIMA
2. rApp decide baseado em CVaR/UE

### 3. Prevenção via Slope

.....

## REGRA 1: PRIORIDADE MÁXIMA

```
if slicer_state == 'CRITICAL':
 return BLOCKED, "Câmeras em risco"
```

## REGRA 2: SLA em risco

```
if cvar_per_ue > 80:
 return BLOCKED, "CVaR $\geq$ 80ms"
```

## REGRA 3: PREVENÇÃO

```
if slope > 2:
 return BLOCKED, "Slope > 2ms/s"
```

## REGRA 4: ECONOMIA segura

```
if cvar_per_ue < 60 and slope < 0:
 return ALLOWED, "Rede estável"
...
```

### *xApp Energy Saver*

```
```c  
  
// Atuador puro - executa comandos do rApp  
void execute_command(char* command) {  
    if (strcmp(command, "FULL_POWER") == 0) {  
        // Potência máxima  
    }  
    if (strcmp(command, "POWER_DOWN") == 0) {  
        // Economia  
    }  
}  
  
// Watchdog - proteção  
void watchdog() {  
    if (no_command_for > 5_seconds) {  
        execute_command("FULL_POWER");  
    }  
}  
...```
```

Métricas de Monitoramento

Data Lake

```
```sql
-- CVaR por UE
SELECT cvar_per_ue_us FROM extended_metrics ORDER BY timestamp DESC LIMIT 10

-- Decisões do rApp
SELECT decision, reason FROM decisions_history ORDER BY timestamp DESC LIMIT 10

-- Latência por UE
SELECT latency_p95_per_ue_us FROM extended_metrics ORDER BY timestamp DESC LIMIT 10
```
```

Dashboard

```
```
URL: http://localhost:5000
Painéis:
■■■ CVaR/UE (44-55ms)
■■■ Slope (+0.0ms/s)
■■■ Decisão rApp
■■■ Regras de Coordenação
```
```

Framework Científico

Referências

```
```
O-RAN Alliance: https://oran-alliance.org
3GPP Standards: TS 28.541 (Network Slicing)
IEEE: "Energy-Efficient O-RAN" papers
```
```

Técnicas Implementadas

```
```
■ Token-based access (JSON commands)
■ Guardrails (CVaR/Slope limits)
■ Watchdog (5s timeout)
■ Priority-based arbitration
■ Trend analysis (Slope)
```
```

Status do Projeto

Implementado

...

- rApp Orchestrator com regras de coordenação
- Dashboard com painéis de coordenação
- xApps como atuadores puros
- Watchdog para proteção
- Análise de tendência (Slope)

...

Em Desenvolvimento

...

- Machine Learning para previsão
- RL para otimização
- Digital Twin para simulação

...

Como Usar

Iniciar Sistema

```
```bash
cd /home/robert/orange_nuclear
./run_greenran_v2.sh
```
```

Acessar Dashboard

...

URL: <http://localhost:5000>

Painéis:

- CVaR/UE (mostra decisão)
- Slope (mostra tendência)
- Decisão rApp
- Regras de Coordenação

...

Monitorar Simulação

```
```bash
./monitor_data_collection.sh
```
```

Contato

Projeto: GreenRAN O-RAN

Instituição: UFPA

Analise do App1 e App2

Objetivo

Este documento resume o estado atual dos dois aplicativos principais do GreenRAN:

- `App1-Vigilancia`
- `App2-Monitoramento`

O foco aqui não é detalhar todo o código, e sim registrar:

- papel de cada app;
- grau de maturidade;
- integração com GreenRAN/hs-3;
- pontos fortes;
- limites atuais.

Localização no código

App1

- backend principal: `apps/app1\_vigilancia/backend/app.py`
- serviços: `apps/app1\_vigilancia/backend/services.py`
- testes: `apps/app1\_vigilancia/tests/test\_app1\_api.py`

App2

- backend principal: `apps/app2\_monitoramento/backend/app.py`
- serviços: `apps/app2\_monitoramento/backend/services.py`
- testes: `apps/app2\_monitoramento/tests/test\_app2\_api.py`

Resumo executivo

| Criterio | App1 | App2 |
|-------------------------------|-------------------------------------|---|
| Objetivo | Vigilancia com cameras e eventos | Monitoramento ambiental e sensores |
| Maturidade | Mais alto | Medio |
| Integracao com GreenRAN | Forte | Boa |
| Dados | Video, eventos, snapshot por camera | Sensores, snapshot ambiental, SLA |
| Estado atual | Mais proximo de produto | Mais proximo de MVP operacional |
| Maior influencia no rApp hoje | Sim | Nao, exceto sob degradacao dos sensores |

App1

O que ele faz

O `App1` representa o caso de uso de vigilancia. Ele concentra:

- cadastro e configuracao de cameras;
- ingestao e organizacao de midia;
- geracao de eventos;
- snapshot operacional por camera;
- exposicao de APIs e interface visual mais rica.

Pontos fortes

- e o app mais completo do projeto;
- tem melhor acabamento funcional;
- conversa melhor com a narrativa principal do GreenRAN;
- hoje e o app que mais pesa nas decisoes do `rApp`;
- possui backend, frontend e testes mais consolidados.

Limites atuais

- a parte de IA ainda esta mais simulada/orquestrada do que um pipeline de visao computacional de producao;
- o app e forte como demonstracao de sistema, mas ainda nao representa um stack real de inferencia pesada ponta a ponta.

App2

O que ele faz

O `App2` representa o monitoramento ambiental/mMTC. Ele concentra:

- snapshot ambiental agregado;
- lista de sensores;
- avaliacao de SLA dos sensores;
- historico e relatorios operacionais.

Pontos fortes

- ficou tecnicamente muito melhor apos a integracao com sensores reais do pipeline `ns-3`;
- hoje ja exporta sensores com semantica coerente;
- o snapshot operacional ficou consistente para o `rApp`;
- a logica de `warning` e `blocked` ja funciona.

Estado atual dos sensores

No estado atual, o `App2` opera com:

- `17` sensores;
- tipos semanticos reais, por exemplo:
- `temperature`
- `humidity`
- `soil\_moisture`

- `soil\_temp`
- `soil\_conductivity`
- conectividades:
- `5g\_native`
- `5g\_redcap`
- gateways:
- `GW-5G-01`
- `GW-5G-02`
- `GW-5G-03`

O snapshot atual do App2 ja reflete um estado coerente de rede e disponibilidade.

Limites atuais

- ainda e um app mais enxuto;
- continua mais focado em monitoramento e SLA do que em analitica mais profunda por sensor;
- tem menos profundidade de produto do que o `App1`.

Relacao com o rApp

Matriz de prioridade e SLA

Hoje a hierarquia pratica do `rApp` e:

1. `camera` (`App1`)
2. `vehicle` (`App3` / trilha veicular)
3. `sensors` (`App2`)
4. economia geral

| Dominio Prioridade Sinais principais Critico Guarda / Warning |
|---|
| --- --- --- --- --- |
| `camera` 1 throughput + latencia throughput `< 25 Mbps` ou latencia `>= 80 ms` throughput e m `25-30 Mbps` ou latencia em `60-80 ms` |
| `vehicle` 2 risco + autonomia + latencia + perda `risk_state=high`, `autonomy_state!=normal`, latencia `>= 100 ms`, perda `>= 5%` `risk_state=medium`, latencia `>= 50 ms`, perda `>= 2%` |
| `sensors` (`App2`) 3 conectividade + entrega + latencia + bateria conectividade `< 85%`, perda `>= 10%`, entrega `< 90%`, latencia `>= 1000 ms`, bateria `< 15%` conectividade `< 90%`, perda `>= 5%`, entrega `< 95%`, latencia `>= 500 ms`, bateria `< 25%` |

Leitura pratica:

- cameras continuam com prioridade maxima;
- veiculos entram logo abaixo das cameras;
- sensores do `App2` continuam relevantes, mas so decidem quando camera e vehicle estao saudaveis.

App1

O `App1` domina a decisao do `rApp` na maior parte dos cenarios atuais porque:

- o cenário principal é orientado a câmeras;
- as regras do orquestrador priorizam proteção de SLA das câmeras.

App2

O `App2` já influencia a decisão quando os sensores degradam. Em particular:

- pode levar o sistema para `warning`;
- pode levar o sistema para `blocked`;
- mas em estado saudável tende a ficar em segundo plano diante das câmeras.

Veredito

App1

- mais maduro;
- mais forte como demonstração principal;
- mais próximo de uma aplicação final do projeto.

App2

- tecnicamente correto e integrado;
- operacionalmente útil;
- ainda mais MVP do que produto final.

Resumo para apresentação

- O `App1-Vigilância` é hoje o app mais maduro do projeto e o principal driver das decisões do `rApp`, porque o cenário atual é centrado em câmeras e SLA de vídeo.
- O `App2-Monitoramento` evoluiu de um mock para uma integração real com sensores vindos do pipeline `ns-3`, com `17` sensores, tipos semânticos corretos e snapshot operacional coerente.
- O `App1` está mais próximo de um produto final; o `App2` está tecnicamente correto e funcional, mas ainda mais próximo de um MVP operacional.
- O `App2` já consegue influenciar o `rApp` em estados de degradação (`warning` e `blocked`), embora no cenário atual as câmeras ainda dominem a lógica de decisão.
- A próxima evolução natural é fortalecer a IA real no `App1` e aprofundar a analítica por sensor no `App2`.

Próximos passos recomendados

App1

- fortalecer a parte de inferência real de vídeo;
- aproximar a camada de IA de um pipeline menos simulado.

App2

- enriquecer análise por sensor;
- introduzir anomalia e correlação temporal mais detalhadas;

- ampliar o peso do App2 em cenários onde sensores também sejam gargalo real.

App3 Veicular - Arquitetura Proposta

Objetivo

Este documento define a arquitetura recomendada para um novo aplicativo:

- `App3-Veicular`

O objetivo do `App3` é representar o domínio de:

- veículos conectados;
- risco operacional;
- autonomia assistida por rede;
- prioridade de SLA para tráfego veicular.

Ele não substitui:

- `App1-Vigilância`
- `App2-Monitoramento`

Ele adiciona um terceiro domínio ao GreenRAN.

Porque um App3

Veículo autônomo/conectado não encaixa bem em:

- `App1`, que é orientado a câmeras, eventos de vigilância e SLA de vídeo;
- `App2`, que é orientado a sensores e monitoramento ambiental.

O `App3` isola uma lógica nova:

- estado por veículo;
- `ego vehicle`;
- risco;
- autonomia;
- relação entre rede e condução;
- eventos de segurança veicular.

Separação entre App e xApp

Arquitetura recomendada:

- `App3-Veicular`
- contexto de alto nível;
- estado dos veículos;
- visualização;
- histórico;

- integracao com CARLA e estado combinado.
- `xApp-VehicleSafety`
- atuacao Near-RT;
- protecao de SLA veicular;
- priorizacao rapida de rede;
- reacao a degradacao critica.

Resumo:

- `App3` pensa o dominio;
- `xApp` reage perto do radio;
- `rApp` arbitra prioridade global.

Posicionamento na arquitetura GreenRAN

```text

CARLA

-> pose, lane, rota, waypoint, risco

ns-3

-> throughput, latencia, perda, handover

csv\_to\_metrics.py

-> unifica contexto fisico + rede

App3-Veicular

-> estado e API veicular

rApp

-> decide prioridade entre camera, vehicle e sensores

xApp-VehicleSafety

-> executa politica de rede para veiculo critico

```

Papel do App3

O `App3` deve concentrar:

- lista de veiculos ativos;
- identificacao do `ego`;
- estado de risco:
- `low`
- `medium`
- `high`
- estado de autonomia:

- `normal`
- `degraded`
- `manual`
- estado de conectividade;
- eventos de segurança veicular;
- histórico curto de degradação.

Dados mínimos por veículo

Campos mínimos recomendados:

- `vehicle\_id`
- `vehicle\_role`
- `imsi`
- `x`
- `y`
- `z`
- `speed\_mps`
- `heading\_deg`
- `lane\_id`
- `waypoint\_id`
- `autonomy\_state`
- `risk\_state`
- `latency\_ms`
- `packet\_loss\_percent`
- `throughput\_mbps`

Campos derivados úteis:

- `is\_ego`
- `network\_state`
- `sla\_state`
- `priority\_required`
- `last\_update\_seconds`

Backend sugerido

Estrutura mínima sugerida:

```
```text
apps/app3_veicular/
backend/
app.py
```

services.py  
schemas.py  
tests/  
test\_app3\_api.py  
...

### **API minima sugerida**

- `GET /api/vehicles`
- lista veiculos ativos
- `GET /api/vehicles/ego`
- estado do ego vehicle
- `GET /api/vehicles/summary`
- resumo operacional
- `GET /api/vehicles/events`
- eventos recentes
- `GET /api/vehicles/health`
- saude do backend

### **Snapshot operacional sugerido**

Arquivo sugerido:

- `/tmp/app3\_veicular/monitoring\_snapshot.json`

Conteudo minimo:

```
```json
{
  "simulation": {
    "source": "carla_ns3_combined",
    "timestamp_iso": "2026-05-08 10:00:00"
  },
  "vehicles": {
    "total_vehicles": 5,
    "ego_present": true,
    "high_risk_vehicles": 0,
    "medium_risk_vehicles": 1,
    "degraded_autonomy_vehicles": 0
  },
  "network": {
    "max_latency_ms": 42.0,
    "max_packet_loss_percent": 0.8
  }
}
```

```

},
"sla": {
  "runtime_status": "ok"
}
}
'''

```

Fonte de dados

No estado atual do projeto, a melhor fonte para o `App3` é:

- `/tmp/xapp\_metrics/extended\_metrics.json`

porque ele já contém:

- `device\_type = vehicle`
- `vehicle\_id`
- `vehicle\_role`
- `speed\_mps`
- `heading\_deg`
- `lane\_id`
- `waypoint\_id`
- `autonomy\_state`
- `risk\_state`
- posicao
- metricas de rede por UE

Ou seja:

- o `App3` não precisa falar direto com o `CARLA` no primeiro momento;
- ele pode nascer lendo o pipeline combinado que já existe.

Hierarquia de prioridade no rApp

A hierarquia recomendada fica:

1. `camera`
2. `vehicle`
3. `app2`
4. economia geral

Matriz resumida de SLA por dominio

Dominio Sinais principais Critico Guarda / Warning
--- --- --- ---
`camera` throughput + latencia throughput `< 25 Mbps` ou latencia `>= 80 ms` throughput em `2 5-30 Mbps` ou latencia em `60-80 ms`

```
| `vehicle` | risco + autonomia + latencia + perda | `risk_state=high`, `autonomy_state!=normal`, latencia `>= 100 ms`, perda `>= 5%` | `risk_state=medium`, latencia `>= 50 ms`, perda `>= 2%` |  
  
| `app2` / sensores | conectividade + entrega + latencia + bateria | conectividade `< 85%`, perda `>= 10%`, entrega `< 90%`, latencia `>= 1000 ms`, bateria `< 15%` | conectividade `< 90%`, perda `>= 5%`, entrega `< 95%`, latencia `>= 500 ms`, bateria `< 25%` |
```

Significado dos termos veiculares

- `risk\_state=high`
 - o veiculo entrou em risco operacional alto;
 - indica um contexto que exige protecao imediata da rede;
 - exemplos futuros:
 - proximidade perigosa;
 - conflito de trajetoria;
 - evento critico de seguranca.
- `autonomy\_state!=normal`
 - o modo de autonomia deixou de estar saudavel;
 - estados esperados:
 - `normal`
 - `degraded`
 - `manual`
 - se estiver diferente de `normal`, o `rApp` trata isso como degradacao do dominio veicular.
- `latencia >= 100 ms`
 - a comunicacao do veiculo esta lenta demais para o servico esperado;
 - isso aumenta o risco de atraso em telemetria, contexto e controle assistido por rede.
- `perda >= 5%`
 - pelo menos `5%` dos pacotes do veiculo estao sendo perdidos;
 - isso indica perda relevante de confiabilidade da comunicacao.

Motivo:

- camera protege SLA de video critico;
- vehicle protege seguranca e autonomia;
- App2 protege continuidade de monitoramento;
- economia entra por ultimo.

Estados de decisao esperados

Para veiculo:

- `VEHICLE\_WARNING`
- `CONDITIONAL`
- `FULL\_POWER\_GUARD`

- `VEHICLE\_CRITICAL`
- `BLOCKED`
- `FULL\_POWER`

Isso ja esta alinhado com a logica atual do `rApp`.

O que vai para o xApp

O `xApp-VehicleSafety` deve receber do `rApp` algo como:

- prioridade de trafego;
- modo protegido;
- politica de latencia;
- reforco de fatia;
- bloqueio de economia agressiva.

O xApp nao precisa conhecer o dominio completo do veiculo. Ele precisa:

- reagir rapido;
- aplicar a politica.

Adaptacao futura para ML e DRL

O dominio veicular pode ser incorporado no pipeline de:

- `ML` do `rApp`
- `DRL` do `rApp`

Adaptacao para ML

Features naturais futuras:

- `total\_vehicles`
- `ego\_present`
- `high\_risk\_vehicles`
- `medium\_risk\_vehicles`
- `degraded\_autonomy\_vehicles`
- `vehicle\_max\_latency\_ms`
- `vehicle\_max\_packet\_loss\_percent`
- `vehicle\_max\_speed\_mps`

Objetivos possiveis:

- prever degradacao veicular antes do estado critico;
- modular a politica de energia com base no risco acumulado;
- aprender combinacoes entre camera, veiculo e sensores.

Adaptacao para DRL

O estado da DRL pode ser expandido com:

- risco agregado dos veiculos;
- latencia maxima do `ego`;
- perda maxima veicular;
- quantidade de veiculos em `warning` ou `critical`;
- contexto combinado `camera + vehicle + app2`.

Objetivos possiveis:

- aprender niveis de energia mais conservadores quando o dominio veicular estiver sob risco;
- evitar degradacao de SLA antes de `VEHICLE\_WARNING` ou `VEHICLE\_CRITICAL`;
- tornar a politica mais sensivel ao `ego vehicle`.

Regra de negocio recomendada

Mesmo com ML e DRL:

1. `camera`
2. `vehicle`
3. `app2`
4. economia geral

Ou seja:

- ML e DRL podem modular a decisao;
- mas nao devem quebrar a prioridade absoluta de camera nem a protecao forte do dominio veicular.

Escopo minimo viavel

Primeira versao do `App3`:

- backend simples;
- leitura de `extended\_metrics.json`;
- snapshot proprio;
- resumo por veiculo;
- um painel do ego vehicle;
- sem UI complexa no inicio.

Proxima implementacao recomendada

Ordem sugerida:

1. criar `apps/app3\_veicular/backend/app.py`
2. criar `apps/app3\_veicular/backend/services.py`

3. gerar `/tmp/app3\_veicular/monitoring\_snapshot.json`
4. expor `GET /api/vehicles/summary`
5. integrar esse snapshot ao `rApp` e ao dashboard

Veredito

Para tratar veiculo autonomo/conectado corretamente no GreenRAN:

- sim, vale criar um novo `App3`;
- sim, tambem vale uma trilha `xApp-VehicleSafety`;
- o `App3` deve nascer em cima do estado combinado `CARLA + ns-3` ja exportado pelo pipeline atual.

CARLA 2D - Modelo Veicular do GreenRAN

Objetivo

Este documento fixa o modelo veicular **2D** que sera usado no projeto para a trilha:

- `CARLA + ns-3 + GreenRAN`

A ideia e:

- usar o `CARLA` como mundo fisico e simulador viario;
- projetar esse mundo em um estado **2D logico**;
- usar esse estado para decisao de rede, risco e prioridade.

Nao e objetivo desta trilha:

- direcao autonoma 3D completa;
- percepcao visual fim a fim;
- sensor fusion pesada;
- planner de conducao industrial.

Porque 2D

Para o GreenRAN, o que mais importa nao e o volume 3D completo da cena. O que importa e:

- onde o veiculo esta;
- para onde vai;
- em que faixa esta;
- qual sua velocidade;
- qual seu risco;
- como esta a rede;
- se precisa prioridade de SLA.

Tudo isso pode ser modelado muito bem com uma camada 2D.

Representacao base

Cada veiculo e descrito no plano por:

- `x`
- `y`
- `heading\_deg`
- `speed\_mps`
- `lane\_id`
- `waypoint\_id`

Campos de estado:

- `vehicle\_id`
- `vehicle\_role`
- `autonomy\_state`
- `risk\_state`

Campos de rede:

- `latency\_ms`
- `packet\_loss\_percent`
- `throughput\_mbps`

Estado minimo por veiculo

Formato minimo recomendado:

```
```json
{
 "vehicle_id": "veh-01",
 "vehicle_role": "ego",
 "x": -120.0,
 "y": -40.0,
 "heading_deg": 18.0,
 "speed_mps": 6.5,
 "lane_id": "lane-west-01",
 "waypoint_id": "wp-west-101",
 "autonomy_state": "normal",
 "risk_state": "low",
 "latency_ms": 18.0,
 "packet_loss_percent": 0.2,
 "throughput_mbps": 6.1
}
```

## Camada de ocupacao 2D

O estado 2D nao precisa ser so uma lista de coordenadas. Ele pode ser visto como:

- posicoes continuas `x,y`, ou
- uma grade 2D de ocupacao.

Versao recomendada para a primeira iteracao:

- manter `x,y` continuos como fonte principal;

- derivar uma grade simples so quando necessario para risco local.

### ***Grade recomendada***

- area operacional projetada no plano;
- grade, por exemplo, `20 x 20`;
- cada celula pode ter:
- livre;
- veiculo;
- pedestre;
- obstaculo;
- area critica;
- cobertura degradada.

### **Entidades observadas**

Na camada 2D, queremos representar:

- `ego vehicle`
- demais veiculos
- pedestres
- obstaculos simples
- zonas viarias relevantes

Em termos de prioridade:

1. `ego vehicle`
2. veiculos em risco
3. trafego ao redor
4. pedestres e obstaculos projetados

### **Estados operacionais**

#### ***Estado de autonomia***

Valores iniciais recomendados:

- `normal`
- `degraded`
- `manual`

#### ***Estado de risco***

Valores iniciais recomendados:

- `low`
- `medium`

- `high`

Heurística inicial:

- `low`
- sem conflito local forte
- rede ok
- `medium`
- degradacao moderada
- risco local intermediario
- `high`
- perigo operacional
- autonomia degradada
- rede insuficiente para operacao segura

## Eventos 2D relevantes

Eventos que a camada 2D deve produzir:

- saida de faixa logica
- aproximacao de obstaculo
- proximidade de pedestre
- degradacao de conectividade em rota critica
- latencia excessiva para o `ego`
- perda acima do limite para veiculo prioritario

Formato sugerido:

```
```json
{
  "event_type": "vehicle_warning",
  "vehicle_id": "veh-01",
  "severity": "warning",
  "reason": "latencia elevada em rota critica",
  "timestamp_iso": "2026-05-08 11:00:00"
}
```
```

## O que vem do CARLA

Do `CARLA`, a camada 2D deve herdar:

- pose do veiculo
- heading

- velocidade
- lane
- waypoint
- mapa viario
- papel do veiculo

Isso ja e suficiente para a primeira versao do GreenRAN veicular.

## O que vem do ns-3

Do `ns-3`, a camada 2D deve receber:

- latencia por veiculo
- perda por veiculo
- throughput por veiculo
- estado de conectividade
- eventuais sinais de degradacao

## Estado combinado

O modelo final usado por `App3`, `xApp-VehicleSafety` e `rApp` deve ser a fusao:

- `CARLA` -> contexto fisico 2D
- `ns-3` -> contexto de rede

Fonte recomendada no estado atual do projeto:

- `/tmp/xapp\_metrics/extended\_metrics.json`

Porque ele ja pode carregar:

- `device\_type = vehicle`
- `vehicle\_id`
- `vehicle\_role`
- `x,y,z`
- `speed\_mps`
- `heading\_deg`
- `lane\_id`
- `waypoint\_id`
- `autonomy\_state`
- `risk\_state`
- metricas de rede

## Uso pelo App3

O `App3-Veicular` deve usar o estado 2D para:

- listar veiculos ativos
- destacar o `ego`
- mostrar risco atual
- mostrar estado de autonomia
- mostrar saude da rede por veiculo
- gerar snapshot operacional

## Uso pelo xApp-VehicleSafety

O `xApp` deve usar o estado 2D para:

- detectar prioridade de rede para veiculo
- pedir protecao de SLA
- sinalizar `warning` ou `critical`
- reagir mais rapido que o `App3`

## Uso pelo rApp

O `rApp` deve usar o estado 2D para arbitrar prioridade global:

1. camera
2. vehicle
3. app2
4. economia geral

Estados esperados:

- `VEHICLE\_WARNING`
- `CONDITIONAL`
- `FULL\_POWER\_GUARD`
- `VEHICLE\_CRITICAL`
- `BLOCKED`
- `FULL\_POWER`

## O que se perde por nao ir para 3D completo

Perdas aceitas nesta estrategia:

- oclusao visual detalhada
- geometria 3D rica
- volume fino dos objetos
- manobras muito complexas
- planejamento visual pesado

Mas isso nao compromete o objetivo principal do projeto:

- rede sensível ao contexto veicular
- prioridade de SLA
- risco operacional
- integração `vehicle <-> ns-3 <-> rApp`

## Escopo mínimo viável

Primeira iteração recomendada:

- `x,y`
- `speed\_mps`
- `heading\_deg`
- `lane\_id`
- `waypoint\_id`
- `autonomy\_state`
- `risk\_state`
- `latency\_ms`
- `packet\_loss\_percent`

Sem:

- percepção visual
- ocupação 3D
- planner de condução completo

## Próxima implementação recomendada

Ordem:

1. consolidar `vehicle` no pipeline atual
2. criar `App3-Veicular`
3. criar `xApp-VehicleSafety`
4. derivar eventos 2D simples de risco
5. ligar esses eventos ao `rApp`

## Veredito

Para o GreenRAN, o modelo `CARLA + 2D` é a escolha mais racional:

- muito mais realista que `mock`;
- muito mais viável que um stack 3D completo;
- suficiente para o objetivo de rede, risco e prioridade veicular.

# Integracao CARLA + ns-3 + GreenRAN

## Objetivo

Este documento define a trilha recomendada para integrar:

- `CARLA` como simulador veicular e ambiental;
- `ns-3` como simulador de rede;
- `GreenRAN` como camada de observabilidade, xApps e decisao do `rApp`.

O objetivo nao e substituir o `ns-3`, e sim complementar o projeto com:

- veiculos dirigidos por rota;
- ambiente viario mais realista;
- telemetria veicular mais rica;
- decisao de rede sensivel ao contexto de conducao.

## Estado atual do repositório

Hoje o projeto ja possui:

- cenario `ns-3` principal em `scenario-greenran.cc`;
- coletor de metricas em `src/csv\_to\_metrics.py`;
- `rApp` em `src/rapp\_orchestrator.py`;
- `App1` e `App2` operacionais;
- carros no `ns-3`, mas ainda como UEs moveis simples, sem autonomia real.

Hoje o projeto **nao possui**:

- bridge com `CARLA`;
- sincronizacao de tempo `CARLA <-> ns-3`;
- ingestao de estado do ego-veiculo a partir do `CARLA`;
- telemetria de controle autonomo integrada ao `rApp`.

## Principio de arquitetura

A arquitetura recomendada e:

1. `CARLA` governa o mundo fisico dos veiculos:

- pose;
- velocidade;
- rota;
- obstaculos;
- atores da cena.

2. `ns-3` governa a rede:

- latencia;
- perda;
- throughput;
- handover;
- qualidade de enlace.

3. o `GreenRAN` governa a decisao:

- protecao de SLA;
- politica de energia;
- priorizacao de trafego;
- interpretacao de contexto veicular.

## Arquitetura minima

```text

CARLA

-> estado dos veiculos

-> eventos de mobilidade e risco

Bridge CARLA/ns-3

-> traduz atores do CARLA em entidades de rede e telemetria

-> sincroniza tick do CARLA com janelas do ns-3

ns-3

-> calcula metricas de rede por veiculo

-> exporta traces

csv_to_metrics.py

-> unifica metricas de rede + contexto veicular

rApp / Data Lake / Apps

-> consomem o estado combinado

-> decidem prioridade, economia e politicas

```

## Modos de execucao

Hoje a integracao suporta dois modos:

- `mock`
- gera `5` veiculos sinteticos;
- e o modo mais estavel para desenvolvimento local.
- `carla`
- consome um servidor `CARLA` real em `host:porta`.

Wrapper principal:

- ``scripts/run_greenran_carla_ns3.sh``

Variaveis uteis:

- ``GREENRAN_CARLA_MODE=mock|carla``
- ``GREENRAN_CARLA_HOST=127.0.0.1``
- ``GREENRAN_CARLA_PORT=2000``
- ``GREENRAN_CARLA_FALLBACK_TO MOCK=1``

Comportamento de fallback:

- se ``GREENRAN_CARLA_MODE=carla`` e o servidor RPC nao responder em ``host:porta``, o wrapper cai automaticamente para ``mock`` por padrao;
- para desabilitar isso e falhar de forma explicita:

```
``bash
```

```
GREENRAN_CARLA_MODE=carla
./scripts/run_greenran_carla_ns3.sh
```

```
``
```

```
GREENRAN_CARLA_FALLBACK_TO MOCK=0
```

## Escopo minimo viavel

### *Fase 1: Veiculos conectados dirigidos por rota*

Entregas:

- usar o ``CARLA`` para gerar:
- posicao;
- heading;
- velocidade;
- waypoint atual;
- estado de obstaculo simples.
- manter o ``ns-3`` como simulador de rede.
- mapear cada veiculo do ``CARLA`` para um ``vehicle_id`` conhecido pelo ``ns-3``.
- exportar metadados de veiculo para o pipeline do ``GreenRAN``.

Status no repositório:

- implementada;
- bridge em ``src/carla_bridge.py``;
- mapeamento em ``src/carla_ns3_mapper.py``;
- wrapper em ``scripts/run_greenran_carla_ns3.sh``;
- enriquecimento do coletor em ``src/csv_to_metrics.py``.

Execucao local recomendada:

```
```bash
./scripts/run_greenran_carla_ns3.sh
```
```

Execucao com CARLA remoto:

```
```bash
GREENRAN_CARLA_MODE=carla \
GREENRAN_CARLA_HOST= \
GREENRAN_CARLA_PORT=2000 \
./scripts/run_greenran_carla_ns3.sh
```
```

Nao inclui ainda:

- controle autonomo completo;
- decisao visual fim a fim;
- fusao de sensores complexa;
- V2X full stack.

## ***Fase 2: Veiculos com risco operacional***

Entregas:

- calcular por veiculo:
- distancia a obstaculo;
- velocidade relativa;
- risco local;
- necessidade de prioridade.
- fazer o `rApp` enxergar `vehicle` como classe propria.
- politicas separadas para:
- camera;
- sensor;
- veiculo.

Status no repositório:

- implementada no `rApp`;
- leitura de `vehicle\_metrics` em `src/rapp\_orchestrator.py`;
- regras:
- `VEHICLE\_CRITICAL` -> `BLOCKED` + `FULL\_POWER`
- `VEHICLE\_WARNING` -> `CONDITIONAL` + `FULL\_POWER\_GUARD`

Validacao rapida sem subir runtime:

```
```bash
python3 ./scripts/test_rapp_vehicle_mock.py
```
```

O script imprime 3 casos:

- `healthy`
- `warning`
- `critical`

### ***Fase 3: Conducao assistida por rede***

Entregas:

- feed de comandos ou alertas para o veiculo;
- prioridade;
- modo seguro;
- degradacao de autonomia;
- perda de conectividade critica.

## **Componentes novos**

### ***1. Bridge CARLA -> GreenRAN***

Arquivo sugerido:

- `src/carla\_bridge.py`

Responsabilidade:

- conectar no servidor do `CARLA`;
- ler atores veiculares;
- serializar um snapshot periodico.

Saida sugerida:

- `/tmp/carla\_state/vehicles.json`

Formato base:

```
```json
{
  "timestamp": 0.0,
  "vehicles": [
    {
      "vehicle_id": "veh-01",
      "role": "ego",
      "x": 0.0,
```

```
"y": 0.0,  
"z": 0.0,  
"speed_mps": 0.0,  
"heading_deg": 0.0,  
"lane_id": "lane-01",  
"waypoint_id": "wp-101",  
"autonomy_state": "normal",  
"risk_state": "low"  
}  
]  
}  
...`
```

2. Mapeador CARLA -> ns-3

Arquivo sugerido:

- `src/carla\_ns3\_mapper.py`

Responsabilidade:

- mapear `vehicle\_id` do `CARLA` para `IMSI`/`UE` do `ns-3`;
- preservar identidade entre sim física e sim de rede.

Saida sugerida:

- `/tmp/carla\_state/vehicle\_network\_map.json`

3. Enriquecimento do coletor

Arquivo a ampliar:

- `src/csv\_to\_metrics.py`

Responsabilidade nova:

- ler o estado do `CARLA`;
- juntar:
 - metricas de rede do `ns-3`
 - contexto fisico do `CARLA`
- exportar `device\_type = vehicle`.

Campos sugeridos por veiculo:

- `vehicle\_id`
- `role`
- `speed\_mps`

- `heading\_deg`
- `lane\_id`
- `autonomy\_state`
- `risk\_state`
- `distance\_to\_obstacle\_m`
- `network\_latency\_ms`
- `packet\_loss\_percent`
- `throughput\_mbps`

4. Políticas no rApp

Arquivo a ampliar:

- `src/rapp\_orchestrator.py`

Responsabilidade nova:

- incluir regras para `vehicle`.

Exemplos:

- veículo em `risk\_state = high`:
- bloquear economia agressiva;
- elevar prioridade de rede.
- latencia alta para veículo crítico:
- forçar `FULL\_POWER`
- e reforçar fatia prioritária.

Estado atual:

- a regra veicular já entra na hierarquia abaixo de `camera` e acima de `App2`;
- snapshots stale geram `warning`;
- `risk\_state=high`, autonomia degradada, latencia alta ou perda alta geram `blocked`.

Sincronizacao recomendada

Não recomendamos sincronizacao forte de simulador por evento logo no inicio.

A primeira versao deve usar sincronizacao fraca por janelas:

- `CARLA` produz snapshots a cada `100-200 ms`;
- `ns-3` continua produzindo traces;
- `csv\_to\_metrics.py` agrega por janela temporal.

Isso simplifica:

- implementacao;
- depuracao;

- estabilidade do runtime.

Modelo operacional recomendado

O que o CARLA fornece

- geometria e dinamica dos veiculos;
- cena viaria;
- rota;
- obstaculos;
- estado do ego-veiculo.

O que o ns-3 fornece

- latencia;
- perda;
- throughput;
- qualidade de enlace;
- impacto de concorrência e congestionamento.

O que o GreenRAN decide

- quando proteger veiculos;
- quando manter economia;
- quando priorizar trafego;
- quando degradar modo autonomo por risco de rede.

Primeira implementacao recomendada

1. criar `carla\_bridge.py`;
2. criar `vehicle\_network\_map.json`;
3. ampliar `csv\_to\_metrics.py` para `device\_type = vehicle`;
4. ampliar `rapp\_orchestrator.py` com regras minimas de veiculo;
5. criar wrapper:
 - `scripts/run\_greenran\_carla\_ns3.sh`

O que nao fazer agora

- nao tentar autonomia visual completa;
- nao tentar replicar `UniAD` no projeto;
- nao tentar sincronizacao forte `tick-by-tick`;
- nao tentar `LoRa/NTN/V2X` completos na primeira iteracao.

Veredito

A integracao `CARLA + ns-3` e viavel e faz sentido para o projeto.

O caminho correto e:

- `CARLA` para mundo fisico dos veiculos;
- `ns-3` para rede;
- `GreenRAN` para politica e decisao.

O primeiro marco tecnico deve ser ****veiculo conectado por rota com contexto combinado de mobilidade + rede****, nao autonomia completa.

`article00`: Resumo Final

Este e o documento executivo da trilha `article00`.

Referencia externa:

- ["/home/robert/Downloads/artigo00.pdf"]()

Documentos tecnicos consolidados:

- [ARTICLE00_METODO_E_EXPERIMENTOS.md]("/home/robert/orange_nuclear/docs/ARTICLE00_METODO_E_EXPERIMENTOS.md:1)

Objetivo

Manter uma trilha experimental separada do GreenRAN operacional para reproduzir, adaptar e comparar a ideia do `artigo00` contra o metodo `ARMD-GreenRAN`.

Essa trilha:

- nao entra no runtime do GreenRAN;
- nao substitui a trilha principal de conflitos do projeto;
- serve para reproducao metodologica, comparacao e figuras de artigo.

Melhor Resultado Geral

Cenario final mais forte do metodo:

- `450 samples`
- `200 epochs`
- `threshold = 0.2`
- `parameter\_kpi\_f1 = 1.0`
- `indirect\_f1 = 1.0`
- `implicit\_f1 = 1.0`

Referencia estavel:

- [article00_final_reference_summary.json]("/home/robert/orange_nuclear/runs/article00/reports/article00_final_reference_summary.json:1)

Melhor Resultado Estrito Em `threshold = 0.5`

Melhor ponto encontrado para comparacao direta mais conservadora:

- `450 samples`
- `200 epochs`
- `hidden\_dim = 32`
- `embed\_dim = 32`

- `dropout = 0.05`
- `temporal\_radius = 3`
- `temporal\_decay = 0.7`
- `fp\_penalty\_weight = 0.3`
- `tp\_reward\_weight = 0.2`
- `hard\_positive\_weight = 0.29`
- `hard\_positive\_focus\_kpis = K2`

Resultado:

- `parameter\_kpi\_f1 = 0.988889`
- `indirect\_f1 = 1.0`
- `implicit\_f1 = 1.0`

Nota Sobre `threshold = 0.9`

Foi feita uma trilha exploratoria separada para tentar melhorar o desempenho em `threshold = 0.9`, sem alterar o setup principal do projeto.

Conclusao:

- houve avancos locais em reconstrucao e em alguns seeds isolados;
- nao houve ganho consistente no conjunto das metricas;
- `0.9` continuou claramente inferior aos cenarios em `0.2` e `0.5`.

Por isso, nenhuma configuracao de `0.9` foi promovida aos graficos ou ao veredito principal da trilha.

Comparacao Com O Artigo

Leitura honesta:

- no melhor cenario geral do metodo, o `ARMD-GreenRAN` supera o artigo;
- no cenario estrito de `threshold = 0.5`, o metodo iguala `indirect` e `implicit`, e fica muito proximo de `1.0` na reconstrucao total;
- a vantagem principal veio de melhor modelagem temporal e melhor funcao de treino, nao de simplesmente aumentar o numero de epochs.

Tabela Comparativa

Metrica	Artigo base	Nosso melhor resultado	Leitura
---	---	---	---
Reconstrucao (`F1 Score`)	`1.0` com `450 samples`, `600 epochs`, `threshold 0.5`	`1.0` com `450 samples`, `200 epochs`, `threshold 0.2`	melhor no cenario geral
Reconstrucao (`F1 Score`) em `threshold 0.5`	`1.0` com `450 samples`, `600 epochs`	`0.988889` com `450 samples`, `200 epochs`	quase igual, mas ainda abaixo
`Indirect`	requer `>= 600 epochs`	`1.0` com `450 samples`, `200 epochs`	melhor que o artigo

```
| `Implicit` | `1.0` com `450 samples`, `200 epochs`, `threshold 0.5` | `1.0` com `450 samples`, `200 epochs` | igualamos o artigo |
```

Resumo da tabela:

- em `indirect`, o `ARMD-GreenRAN` supera o artigo em eficiencia de treino;
- em `implicit`, o metodo chega ao mesmo teto do melhor caso do artigo;
- em reconstrucao geral, o melhor cenario do metodo supera o artigo;
- em reconstrucao estrita com `threshold 0.5`, o metodo fica muito proximo, mas ainda abaixo do `1.0`.

O Que Melhorou O Metodo

Os ganhos em apenas `200 epochs` vieram principalmente de:

- janela temporal expandida;
- perda com penalizacao de falso positivo;
- recompensa de verdadeiro positivo;
- hard-positive focado em `K2` no caso estrito de `0.5`;
- selecao composta de checkpoint;
- capacidade maior do encoder no melhor ponto de `0.5`.

Onde Estao Os Artefatos

Relatorios:

- `[runs/article00/reports](/home/robert/orange_nuclear/runs/article00/reports)`

Figuras comparativas:

- `[runs/article00/comparison_figures](/home/robert/orange_nuclear/runs/article00/comparison_figures)`

Pasta organizada para artigo:

- `[graficos_selecionados_artigo](/home/robert/orange_nuclear/runs/article00/graficos_selecionados_artigo)`

Nessa pasta, os graficos foram separados por metrica:

- `reconstrucao`
- `indirect`
- `implicit`

E, dentro de cada metrica, por:

- `threshold 0.2`
- `threshold 0.5`
- `threshold 0.9`
- `dataset 50`
- `dataset 150`
- `dataset 450`

Observacao importante:

- [aggregate_report.json](/home/robert/orange_nuclear/runs/article00/reports/aggregate_report.json:1)

reflete a ultima corrida executada e pode estar em `50`, `150` ou `450 samples`;

- o resumo estavel do melhor cenario final fica em

[article00_final_reference_summary.json](/home/robert/orange_nuclear/runs/article00/reports/article00_final_reference_summary.json:1).

Regra De Interpretacao Dos Graficos

- quando o grafico e `Epochs for Threshold(...)` , o ponto de referencia e `Random`;
- quando o grafico e `Epochs for Dataset(...)` , o ponto de referencia e `No Threshold`.

Status

Status atual da trilha:

- experimental forte;
- separada do GreenRAN;
- com artefatos e comparativos consolidados;
- ainda nao homologada como reproducao metodologica final do paper.

`article00`: Metodo E Experimentos

Este e o documento tecnico consolidado da trilha `article00`.

Resumo executivo:

- [ARTICLE00_RESUMO_FINAL.md](/home/robert/orange_nuclear/docs/ARTICLE00_RESUMO_FINAL.md:1)

Referencia externa:

- [/home/robert/Downloads/artigo00.pdf]()

Escopo

A trilha `article00` foi criada para:

- gerar dataset sintetico temporal;
- treinar um `GraphSAGE` temporal isolado;
- reconstruir relacoes `P-K`;
- rotular conflitos `direct`, `indirect` e `implicit`;
- produzir comparativos com `Random`, `GraphSAGE-CL` e `ARMD-GreenRAN`.

Ela nao integra o runtime do GreenRAN.

Pipeline

Arquivos principais:

- gerador de dataset:
- [generate_article00_dataset.py](/home/robert/orange_nuclear/scripts/generate_article00_dataset.py:1)
- trainer:
- [train_graphsage_article00.py](/home/robert/orange_nuclear/training/train_graphsage_article00.py:1)
- runner:
- [run_article00_experiments.py](/home/robert/orange_nuclear/scripts/run_article00_experiments.py:1)
- gerador de figuras:
- [generate_graphsage_article00_figures.py](/home/robert/orange_nuclear/scripts/generate_graphsage_article00_figures.py:1)
- gerador dos comparativos:
- [generate_article00_armd_comparison_figures.py](/home/robert/orange_nuclear/scripts/generate_article00_armd_comparison_figures.py:1)

Estrutura De Saida

```
```text
```

```
runs/article00/
```

```
■■■ datasets/
```

```
■■■ training/
```

■■■ reports/  
■■■ figures/  
■■■ comparison\_figures/  
...

## Principais Mudancas Que Fizeram O Metodo Convergir

### 1. Modelagem temporal melhor

Os melhores cenarios usaram:

- ``temporal_radius = 2`` ou ``3``
- ``temporal_decay = 0.7``

Isso deu mais contexto temporal para separar melhor as relacoes do grafo.

### 2. Perda mais informada

Entraram termos para:

- penalizar falso positivo;
- recompensar verdadeiro positivo;
- forçar arestas reais criticas a cruzarem o threshold no caso mais dificil.

No caso estrito de ``threshold = 0.5``, o gargalo residual estava concentrado em arestas ligadas ao ``K2``, especialmente no ``seed 43``. O uso de ``hard_positive_focus_kpis = K2`` foi o passo que destravou esse caso.

### 3. Selecao composta de checkpoint

Em vez de selecionar por uma metrica so, passou-se a usar:

- `reconstrucao`;
- ``indirect``;
- ``implicit``.

Isso evitou otimizar demais um alvo e perder o resto.

### 4. Capacidade do encoder

O melhor cenario estrito em ``0.5`` apareceu com:

- ``hidden_dim = 32``
- ``embed_dim = 32``
- ``dropout = 0.05``

## Protocolo Experimental Consolidado

### Melhor cenario geral

- `450 samples`
- `200 epochs`
- `threshold = 0.2`
- `1.0 / 1.0 / 1.0`

Referencia estavel:

- [article00\_final\_reference\_summary.json](/home/robert/orange\_nuclear/runs/article00/reports/article00\_final\_reference\_summary.json:1)

### **Melhor cenario estrito em `0.5`**

- `450 samples`
- `200 epochs`
- `hidden\_dim = 32`
- `embed\_dim = 32`
- `dropout = 0.05`
- `temporal\_radius = 3`
- `temporal\_decay = 0.7`
- `fp\_penalty\_weight = 0.3`
- `tp\_reward\_weight = 0.2`
- `hard\_positive\_weight = 0.29`
- `hard\_positive\_focus\_kpis = K2`
- `0.988889 / 1.0 / 1.0`

### **Comparacao consolidada com o artigo**

Metrica	Artigo base	ARMD-GreenRAN	Leitura
---	---	---	---
Reconstrucao geral	`1.0` com `450 samples`, `600 epochs`, `threshold 0.5`	`1.0` com `450 samples`, `200 epochs`, `threshold 0.2`	melhor cenario geral do metodo
Reconstrucao estrita em `0.5`	`1.0` com `450 samples`, `600 epochs`	`0.988889` com `450 samples`, `200 epochs`	quase igual
`Indirect`	`>= 600 epochs` para `1.0`	`1.0` com `450 samples`, `200 epochs`	melhor em eficiencia
`Implicit`	`1.0` com `450 samples`, `200 epochs`, `threshold 0.5`	`1.0` com `450 samples`, `200 epochs`	igualado

### **Corridas adicionais de dataset**

Tambem foram rodadas variantes para:

- `50 samples`
- `150 samples`
- `450 samples`

Esses artefatos alimentam os comparativos por `Dataset`.

### **Exploracao adicional de `threshold = 0.9`**

Foi aberta uma trilha separada em `runs/article00\_threshold\_09/` para testar melhorias especificas de `threshold = 0.9`, sem sobrescrever a trilha principal de `article00`.

O resultado final dessa exploracao foi:

- melhora parcial em reconstrucao em alguns candidatos;
- melhora parcial em `implicit` em alguns candidatos;
- ausencia de ganho consistente no agregado completo;
- nenhuma configuracao suficientemente forte para substituir os cenarios principais em `0.2` ou `0.5`.

Decisao final:

- manter `0.9` apenas como referencia de sensibilidade mais rigida;
- nao promover esses runs aos docs/graficos principais.

## Graficos Consolidados

Graficos comparativos principais:

- `[runs/article00/comparison_figures]/(home/robert/orange_nuclear/runs/article00/comparison_figures)`

Pasta organizada para consulta:

- `[graficos_selecionados_artigo]/(home/robert/orange_nuclear/runs/article00/graficos_selecionados_artigo)`

Separacao principal:

- `reconstrucao`
- `indirect`
- `implicit`

Dentro de cada metrica:

- `threshold 0.2`
- `threshold 0.5`
- `threshold 0.9`
- `dataset 50`
- `dataset 150`
- `dataset 450`

Regra dos pontos de referencia:

- `Epochs for Threshold(...)` usa `Random`;
- `Epochs for Dataset(...)` usa `No Threshold`.

Observacao sobre relatorios:

- [aggregate\_report.json](/home/robert/orange\_nuclear/runs/article00/reports/aggregate\_report.json:1)

e [article00\_run\_multiseed.json](/home/robert/orange\_nuclear/runs/article00/reports/article00\_run\_multiseed.json:1)

refletem a ultima corrida executada;

- para citar o melhor cenario consolidado da trilha, usar

[article00\_final\_reference\_summary.json](/home/robert/orange\_nuclear/runs/article00/reports/article00\_final\_refere  
nce\_summary.json:1).

## Limites

Mesmo com o desempenho forte, a trilha continua:

- experimental;
- separada do GreenRAN operacional;
- ainda sem homologacao metodologica final do paper.

Ela ja e suficiente para comparacao experimental forte, mas nao deve ser vendida como reproducao metodologica final sem a etapa de homologacao.

# NS3 Article00 Temporal Pipeline

Este fluxo prepara o `ns-3` para rodar a mesma metodologia temporal usada na trilha sintética `article00`, mas sobre dados coletados do runtime GreenRAN.

## O que o pipeline faz

1. sobe `nearRT-RIC + ns-3 + csv\_to\_metrics + rApp + apps`;
2. coleta conflitos por cenário com `run\_conflict\_experiments.py`;
3. converte o dataset coletado para o formato temporal do `article00`;
4. treina o `train\_graphsage\_article00.py` no dataset convertido;
5. agrega os resultados multi-seed e gera figuras.

## Wrapper pronto

O wrapper principal é:

- `[run_ns3_article00_pipeline.sh](/home/robert/orange_nuclear/scripts/run_ns3_article00_pipeline.sh:1)`

## Comando direto

```
``bash
./scripts/run_ns3_article00_pipeline.sh \
--scenario conflito_implicito \
--rounds 10 \
--duration 120 \
--samples 450 \
--seeds 42,43,44,45,46,47
``
```

Valores suportados para `--samples`:

- `50`
- `150`
- `450`
- `0` para usar o export completo

## Artefatos

Coleta bruta:

- `runs/experimentos\_conflitos/\_conflict\_protocol`

Trilha temporal convertida:

- `runs/ns3\_article00/datasets/samples\_`

- ``runs/ns3_article00//training``
- ``runs/ns3_article00//reports``
- ``runs/ns3_article00//figures``

## Scripts envolvidos

- `[generate_ns3_article00_dataset.py](/home/robert/orange_nuclear/scripts/generate_ns3_article00_dataset.py:1)`
- `[run_ns3_article00_experiments.py](/home/robert/orange_nuclear/scripts/run_ns3_article00_experiments.py:1)`
- `[run_ns3_article00_pipeline.sh](/home/robert/orange_nuclear/scripts/run_ns3_article00_pipeline.sh:1)`
- `[train_graphsage_article00.py](/home/robert/orange_nuclear/training/train_graphsage_article00.py:1)`

## Observacao metodologica

Esse fluxo reutiliza o trainer temporal do ``article00``, mas os parametros do ``ns-3`` sao convertidos para sinais proxy derivados dos eventos coletados (``power``, ``confidence``, ``mitigation``, ``severity``). Ou seja: o pipeline esta pronto para rodar, mas a calibracao final desses proxies ainda e parte da validacao experimental.